

Software Factory and CI/CD

Version 1, 2026-07-10

Introduction

Welcome to this quick course on the **Software Factory and CI/CD**. This course is the **third** in a series of **six** courses about **Application Platform Delivery**:

- *Foundations: Developer Experience and GitOps Workflows*
- *Cloud-Native Development with Dev Spaces*
- *Software Factory and CI/CD(This course)*
- *Application Modernization*
- *Developer Hub and Platform Engineering*
- *Trusted Software Supply Chain (TSSC)*

What's in this course?

By the end of this module, you will be able to deploy, configure, and use Red Hat OpenShift Dev Spaces to develop, build, and deploy applications through integrated GitLab and CI/CD workflows.

Duration: 2.5 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- .. (SME)
- .. (SME)
- Yogita Soni (Content Architect)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

Based on the section index, here are concise learning objectives:

Learning Objectives

By the end of this module, you will be able to:

- Describe the purpose and architecture of Red Hat OpenShift Dev Spaces.
- Explain the key concepts and core components of Dev Spaces.
- Understand how Dev Spaces fits into a Software Factory and supports cloud-native application development.

- Identify the supporting services required to enable an integrated developer platform.
- Deploy and configure the Red Hat OpenShift Dev Spaces Operator on an OpenShift cluster.
- Integrate Dev Spaces with GitLab to enable source code management and developer workflows.
- Create and use Dev Spaces workspaces for cloud-native application development.
- Understand how Continuous Integration (CI) pipelines are triggered and executed from developer workspaces.
- Deploy applications to OpenShift using automated delivery workflows.
- Explain the role of promotion pipelines in advancing applications across development, testing, and production environments.

Prerequisites

This course assumes that you have the following prior experience:

1. [Red Hat OpenShift Container Platform \(OCP\) Technical Overview](#) or Kubernetes Basics

Lab Environment

Click the [RHDP](#) link above, log in to RHDP, and click **Order** to request a new [Service Enablement: App Platform Software Factory](#). On the order page for the catalog item, do the following:

- **Activity:** Select **Practice/Enablement**
- **Purpose:** Select **Learning about the Product**
- **Salesforce ID:** If you have an opportunity ID, enter it into this field, otherwise select **Project**, and then enter "Learning about RHCL" in the text field. You will see a warning message that a Salesforce ID is required, but you can safely ignore it.
- **Features:** Select **Enable Let's Encrypt**
- **Region:** Select the AWS region closest to your location
- **User count:** **1**
- **Control Plane Count:** **1**
- **OpenShift Version:** **4.16**
- **Control Plane Instance Type:** **m6a.4xlarge**
- Select the checkbox at the bottom of the page to confirm that lab costs will be charged to your cost center, and click **Order**.



The lab environment will take an hour to provision. You can check the status and details of the OpenShift cluster from the **Services** page of RHDP. The classroom is available for 48 hours initially. You can extend the duration of classroom availability by clicking **Help > Get Technical Help** in RHDP, and then requesting an extension by opening a ServiceNow ticket with the RHDP team.

RHDP Portal Links

- [RedHat Associates](#)
- [RedHat Partners](#)

Software Factory Overview

Overview

Modern software development requires teams to deliver applications rapidly without compromising quality, security, or reliability. A Software Factory provides a standardized platform that automates the software delivery lifecycle, enabling developers to focus on writing code while the platform handles repetitive operational tasks.

A Software Factory combines development tools, CI/CD automation, security scanning, artifact management, GitOps, and platform services into an integrated ecosystem. Instead of each team building and maintaining its own delivery process, the Software Factory offers reusable capabilities that promote consistency and reduce operational overhead.

A typical Software Factory includes source code management, developer workspaces, continuous integration pipelines, container registries, artifact repositories, security and compliance tools, GitOps-based deployment, and an application platform such as Red Hat OpenShift.

By adopting a Software Factory, organizations can standardize development practices, accelerate application delivery, improve collaboration between development and platform teams, and ensure applications are deployed securely and consistently across environments.

Key Benefits

- Standardized development workflows
- Faster software delivery
- Improved collaboration
- Automated quality and security checks
- Consistent deployments across environments
- Reduced operational complexity

Lab 1

Overview

Deploy and configure the platform services required by the application and CI/CD pipeline: PostgreSQL database for application data persistence, AMQ Streams (Kafka) for messaging, OpenShift Pipelines (Tekton) for CI, and the OpenTelemetry Operator for distributed tracing.

These services will be shared across your development workflow in Dev Spaces and production deployments.



HashiCorp Vault is already deployed as part of the infrastructure bootstrap and does not require student installation.

HashiCorp Vault (Pre-Deployed)

HashiCorp Vault provides centralized secrets management for the platform. It has been pre-deployed by the infrastructure bootstrap process and is ready to use.

- **URL:** {vault_url}[^]
- **Authentication Method:** OIDC
 - `platform-admin` / `openshift` (administrator access)
 - `platform-developer` / `openshift` (developer access)
- **Purpose:** CI/CD pipeline credentials and application secrets

The Vault instance is already configured with:

- KV v2 secrets engine enabled at `secret/`
- Kubernetes authentication method for pipeline ServiceAccounts
- Registry credentials stored at `secret/registry`
- RBAC policies for pipeline access

Your Tekton pipelines can authenticate to Vault using their ServiceAccount tokens to retrieve registry credentials at runtime.



Access the Vault UI at {vault_url}[^] and log in with the OIDC method to explore the pre-configured secrets. If prompted for a role, leave it blank or use the default.

Install OpenShift Pipelines (Tekton)

OpenShift Pipelines provides the Tekton-based CI engine used to build, test, and push container images.

Install via OperatorHub

1. Open {openshift_console_url}[^] with an admin user.
2. In the OpenShift Web Console, go to menu:Ecosystem[Software Catalog]
3. Search for **Red Hat OpenShift Pipelines**
4. Click btn:[Install]
5. Accept the defaults:
 - **Update channel:** latest
 - **Installation mode:** All namespaces
 - **Installed Namespace:** `openshift-operators`
6. Click btn:[Install]
7. Wait for the operator to reach the `Succeeded` phase

Verify from the CLI:

```
oc get csv -n openshift-operators | grep pipelines
```

You should see the Pipelines operator listed with phase **Succeeded**.

Verify Tekton Components

Confirm the Tekton controller pods are running:

```
oc get pods -n openshift-pipelines
```

You should see pods for **tekton-pipelines-controller**, **tekton-triggers-controller**, and related components.

Install AMQ Streams (Apache Kafka)

AMQ Streams provides Apache Kafka on OpenShift. The ETX App Base application uses Kafka for messaging between services.

Install via OperatorHub

1. Open {openshift_console_url}[^] with an admin user.
2. In the OpenShift Web Console, go to menu:Ecosystem[Software Catalog]
3. Search for **Streams for Apache Kafka**



Do **NOT** install the "Streams for Apache Kafka Proxy" operator. Only install "Streams for Apache Kafka" (AMQ Streams).

4. Click btn:[Install]
5. Accept the defaults:
 - **Update channel:** stable
 - **Installation mode:** All namespaces
 - **Installed Namespace:** **openshift-operators**
6. Click btn:[Install]
7. Wait for the operator to reach the **Succeeded** phase

Verify from the CLI:

```
oc get csv -n openshift-operators | grep amqstreams
```

You should see the AMQ Streams operator listed with phase **Succeeded**.

Deploy Kafka Cluster

Now that the operator is installed, deploy a Kafka cluster in your application namespace.

Starting with AMQ Streams 3.x, Kafka clusters use **KafkaNodePools** to manage broker nodes. A **KafkaNodePool** defines a group of Kafka nodes with specific roles (controller, broker, or both) and their storage configuration. This deployment uses a single node pool with combined controller and broker roles, running in **KRaft mode** (without ZooKeeper).

```
oc apply -f - <<'EOF'
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool
metadata:
  name: kafka
  namespace: etx-app-dev
  labels:
    strimzi.io/cluster: parasol-kafka
spec:
  replicas: 1
  roles:
    - controller
    - broker
  storage:
    type: ephemeral
---
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: parasol-kafka
  namespace: etx-app-dev
  annotations:
    strimzi.io/node-pools: enabled
    strimzi.io/kraft: enabled
spec:
  kafka:
    version: 4.2.0
    metadataVersion: 4.2-IV0
    listeners:
      - name: plain
        port: 9092
        type: internal
        tls: false
      - name: tls
        port: 9093
        type: internal
        tls: true
    config:
      offsets.topic.replication.factor: 1
      transaction.state.log.replication.factor: 1
      transaction.state.log.min.isr: 1
      default.replication.factor: 1
```



```
    min.insync.replicas: 1
  entityOperator:
    topicOperator: {}
    userOperator: {}
EOF
```

Wait for the Kafka cluster to be ready:

```
oc wait kafka/parasol-kafka --for=condition=Ready --timeout=300s -n etx-app-dev
```

Verify the Kafka pods are running:

```
oc get pods -l strimzi.io/cluster=parasol-kafka -n etx-app-dev
```

You should see: - **parasol-kafka-kafka-0** — Kafka broker (running in KRaft mode with controller + broker roles) - **parasol-kafka-entity-operator-*** — Topic and User operators

Get the Kafka bootstrap server:

```
oc get service parasol-kafka-kafka-bootstrap -n etx-app-dev -o
jsonpath='{.metadata.name}:{.spec.ports[0].port}{"\n"}'
```

This returns **parasol-kafka-kafka-bootstrap:9092** which will be used to connect to Kafka from your application.

Create Kafka Topics

Create the topics required by the ETX App Base application:

```
oc apply -f - <<'EOF'
apiVersion: kafka.strimzi.io/v1
kind: KafkaTopic
metadata:
  name: intake
  namespace: etx-app-dev
  labels:
    strimzi.io/cluster: parasol-kafka
spec:
  partitions: 1
  replicas: 1
  config:
    retention.ms: 604800000
    segment.bytes: 1073741824
EOF
```

Verify the topic was created:

```
oc get kafkatopic -n etx-app-dev
```

Deploy PostgreSQL Database

Deploy PostgreSQL using the Red Hat certified container image for application data persistence.

Create PostgreSQL Deployment

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: Secret
metadata:
  name: parasol-db-secret
  namespace: etx-app-dev
type: Opaque
stringData:
  database-name: parasol
  database-user: parasol
  database-password: parasol
---
apiVersion: v1
kind: Service
metadata:
  name: parasol-db
  namespace: etx-app-dev
spec:
  ports:
    - name: postgresql
      port: 5432
      targetPort: 5432
  selector:
    app: parasol-db
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: parasol-db
  namespace: etx-app-dev
spec:
  replicas: 1
  selector:
    matchLabels:
      app: parasol-db
  template:
    metadata:
      labels:
        app: parasol-db
    spec:
      containers:
```

```

- name: postgresql
  image: registry.redhat.io/rhel9/postgresql-16:latest
  ports:
    - containerPort: 5432
  env:
    - name: POSTGRESQL_USER
      valueFrom:
        secretKeyRef:
          name: parasol-db-secret
          key: database-user
    - name: POSTGRESQL_PASSWORD
      valueFrom:
        secretKeyRef:
          name: parasol-db-secret
          key: database-password
    - name: POSTGRESQL_DATABASE
      valueFrom:
        secretKeyRef:
          name: parasol-db-secret
          key: database-name
  volumeMounts:
    - name: postgresql-data
      mountPath: /var/lib/pgsql/data
  livenessProbe:
    exec:
      command:
        - /usr/libexec/check-container
        - --live
    initialDelaySeconds: 120
    timeoutSeconds: 10
  readinessProbe:
    exec:
      command:
        - /usr/libexec/check-container
    initialDelaySeconds: 5
    timeoutSeconds: 1
  resources:
    limits:
      memory: 512Mi
      cpu: 500m
    requests:
      memory: 256Mi
      cpu: 100m
  volumes:
    - name: postgresql-data
      emptyDir: {}

```

EOF



This Deployment uses `emptyDir` for storage, which means data is lost when the pod restarts. For production, use a `PersistentVolumeClaim` backed by persistent storage.

Wait for PostgreSQL to be ready:

```
oc rollout status deployment/parasol-db -n etx-app-dev
```

Verify the PostgreSQL pod is running:

```
oc get pods -l app=parasol-db -n etx-app-dev
```

Test the database connection:

```
oc run postgresql-test --rm -it --restart=Never \
  --image=registry.redhat.io/rhel9/postgresql-16:latest \
  --env PGPASSWORD=$(oc get secret -n etx-app-dev parasol-db-secret -o
jsonpath='{.data.database-password}' | base64 -d) \
  -n etx-app-dev -- \
  psql -h parasol-db \
  -U $(oc get secret -n etx-app-dev parasol-db-secret -o jsonpath='{.data.database-
user}' | base64 -d) \
  -d $(oc get secret -n etx-app-dev parasol-db-secret -o jsonpath='{.data.database-
name}' | base64 -d) \
  -c '\conninfo'
```

You should see connection information confirming PostgreSQL is accessible.



The database credentials are stored in the `parasol-db-secret` Secret. Your application will use these same credentials to connect.

Install the OpenTelemetry Operator

The OpenTelemetry Operator manages the deployment of OpenTelemetry Collectors for distributed tracing and metrics collection.

Install via OperatorHub

1. Open {openshift_console_url}[^] with an admin user.
2. In the OpenShift Web Console, go to menu:Ecosystem[Software Catalog]
3. Search for **Red Hat build of OpenTelemetry**
4. Click btn:[Install]
5. Accept the defaults:
 - **Update channel:** stable
 - **Installation mode:** All namespaces
 - **Installed Namespace:** `openshift-opentelemetry-operator`
6. Click btn:[Install]

7. Wait for the operator to reach the **Succeeded** phase

Verify from the CLI:

```
oc get csv -n openshift-operators | grep opentelemetry
```

You should see the OpenTelemetry operator listed with phase **Succeeded**.

Deploy an OpenTelemetry Collector

Create a collector instance in your application namespace. This collector will receive traces from instrumented applications and export them to the console for demonstration purposes:

```
oc apply -f - <<'EOF'
apiVersion: opentelemetry.io/v1beta1
kind: OpenTelemetryCollector
metadata:
  name: otel-collector
  namespace: etx-app-dev
spec:
  mode: deployment
  config:
    receivers:
      otlp:
        protocols:
          grpc:
            endpoint: 0.0.0.0:4317
          http:
            endpoint: 0.0.0.0:4318
    processors:
      batch: {}
      probabilistic_sampler:
        sampling_percentage: 100.0
    exporters:
      debug:
        verbosity: detailed
  service:
    pipelines:
      traces:
        receivers: [otlp]
        processors: [probabilistic_sampler, batch]
        exporters: [debug]
EOF
```



verbosity: detailed prints full trace details to stdout (visible in collector pod logs). The **logging** exporter was removed in Collector v0.111+; use **debug** instead.



For production, replace the **debug** exporter with a real tracing backend like Tempo,

Jaeger, or a cloud provider's tracing service.

Verify the collector pod is running:

```
oc get pods -l app.kubernetes.io/name=otel-collector-collector -n etx-app-dev
```

Enable Auto-Instrumentation

Create an **Instrumentation** resource to enable automatic trace injection for Java applications:

```
oc apply -f - <<'EOF'
apiVersion: opentelemetry.io/v1alpha1
kind: Instrumentation
metadata:
  name: java-instrumentation
  namespace: etx-app-dev
spec:
  exporter:
    endpoint: http://otel-collector-collector:4317
  propagators:
    - tracecontext
    - baggage
  sampler:
    type: always_on
  java:
    image: ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-
java:latest
EOF
```



The **sampler.type: always_on** configuration ensures all traces are captured, which is appropriate for development and lab environments. In production, you might use **parentbased_traceidratio** to sample a percentage of traces.

To instrument a deployment, add the following annotation to its pod template:

```
metadata:
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
```

This will be used when deploying the application in [Application Deployment](#).

Summary

You now have the following services deployed and ready:

- **Vault** (pre-deployed) — secrets management with Kubernetes authentication for pipeline access

- **OpenShift Pipelines** — Tekton CI engine ready to run pipelines
- **AMQ Streams Kafka Cluster** — `parasol-kafka-kafka-bootstrap:9092` with `intake` topic
- **PostgreSQL Database** — `parasol-db:5432` with database `parasol`
- **OpenTelemetry** — collector and auto-instrumentation configured for distributed tracing

Your application can now connect to these services:

- **Kafka bootstrap server:** `parasol-kafka-kafka-bootstrap:9092`
- **PostgreSQL host:** `parasol-db:5432`
- **Database credentials:** Available in `parasol-db-secret` Secret

Next Steps

Proceed to [Dev Spaces & IDE Setup](#) to create your development workspace and connect to these services.

Lab 2

Overview

Deploy your own Argo CD instance to manage application deployments using a GitOps workflow. The CI pipeline from the previous section pushes container images; Argo CD watches a GitOps repository and syncs Kubernetes manifests to the cluster.



You will deploy your **own** Argo CD instance in the `etx-app-dev` namespace. The pre-deployed OpenShift GitOps operator in `openshift-gitops` is used for infrastructure automation only and is separate from your application deployment workflows.

Install OpenShift GitOps Operator

The OpenShift GitOps operator is already installed cluster-wide from the infrastructure setup. Verify it's available:

```
oc get csv -n openshift-operators | grep gitops
```

You should see `openshift-gitops-operator` with phase `Succeeded`.

Configure Keycloak OIDC for Argo CD

The ETX platform uses a shared Keycloak OIDC client (`argocd`) for all ArgoCD instances. This provides centralized authentication with the same credentials across GitLab, Quay, Vault, and ArgoCD.

Create ArgoCD OIDC Secret

The OIDC client **argocd** already exists in Keycloak realm **etx-lab**. Create a secret with the shared credentials:

```
# Get Keycloak URL (etx-sso route)
KEYCLOAK_URL="https://etx-sso.apps.$(oc get route -n keycloak -o
jsonpath='{.items[0].spec.host}' | sed 's/sso\.apps\.//')"
```

```
# Use the shared argocd client secret from etx-gitops
CLIENT_SECRET="abJIhC36Q7DPscCUNNu27QKSdF9euOpIr16e7VvM"
```

```
oc create secret generic argocd-oidc-secret -n etx-app-dev \
--from-literal=clientSecret=${CLIENT_SECRET}
```

```
# Add labels to help ArgoCD operator recognize the secret
oc label secret argocd-oidc-secret -n etx-app-dev \
app.kubernetes.io/name=argocd-oidc-secret \
app.kubernetes.io/part-of=argocd
```

```
echo "Keycloak issuer: ${KEYCLOAK_URL}/realms/etx-lab"
```



The ArgoCD OIDC client is shared across all ArgoCD instances in the ETX platform. Do not create a new client in Keycloak. Use the existing **argocd** client with the provided secret.

Deploy Your Argo CD Instance

Create an Argo CD instance in your namespace for managing application deployments.

First, get the cluster domain dynamically:

```
# Get cluster domain from existing Keycloak route
CLUSTER_DOMAIN="$(oc get route -n keycloak -o jsonpath='{.items[0].spec.host}' | sed
's/sso\.apps\.//')"
```

```
echo "Cluster domain: ${CLUSTER_DOMAIN}"
```

Then create the ArgoCD instance with OIDC configuration:

```
CLUSTER_DOMAIN="$(oc get route -n keycloak -o jsonpath='{.items[0].spec.host}' | sed
's/sso\.apps\.//')"
```

```
cat <<EOF | oc apply -f -
apiVersion: argoproj.io/v1beta1
kind: ArgoCD
metadata:
  name: argocd
```



```

namespace: etx-app-dev
spec:
  server:
    autoscale:
      enabled: false
    grpc:
      ingress:
        enabled: false
    ingress:
      enabled: false
    insecure: true
    resources:
      limits:
        cpu: 500m
        memory: 256Mi
      requests:
        cpu: 125m
        memory: 128Mi
    route:
      enabled: true
      tls:
        termination: edge
        insecureEdgeTerminationPolicy: Redirect
    service:
      type: ClusterIP
  grafana:
    enabled: false
    ingress:
      enabled: false
    route:
      enabled: false
  prometheus:
    enabled: false
    ingress:
      enabled: false
    route:
      enabled: false
  initialSSHKnownHosts: {}
  rbac:
    defaultPolicy: 'role:readonly'
    policy: |
      g, platform-admin, role:admin
      g, platform-developer, role:admin
    scopes: '[groups, preferred_username]'
  extraConfig:
    oidc.config: |
      name: ETX Keycloak
      issuer: https://etx-sso.apps.${CLUSTER_DOMAIN}/realms/etx-lab
      clientID: argocd
      clientSecret: \${argocd-oidc-secret:clientSecret}
      requestedScopes:

```

```

    - openid
  requestedIDTokenClaims:
    groups:
      essential: true
  logoutURL: https://etx-sso.apps.${CLUSTER_DOMAIN}/realms/etx-
lab/protocol/openid-connect/logout
  repo:
    resources:
      limits:
        cpu: 1000m
        memory: 1Gi
      requests:
        cpu: 250m
        memory: 256Mi
  resourceExclusions: |
    - apiGroups:
      - tekton.dev
      clusters:
      - '*'
      kinds:
      - TaskRun
      - PipelineRun
  controller:
    processors: {}
    resources:
      limits:
        cpu: 2000m
        memory: 2Gi
      requests:
        cpu: 250m
        memory: 1Gi
  ha:
    enabled: false
    resources:
      limits:
        cpu: 500m
        memory: 256Mi
      requests:
        cpu: 250m
        memory: 128Mi
EOF

```

Expected output:

```
argocd.argoproj.io/argocd created
```



The `server.insecure: true` setting configures Argo CD to serve HTTP traffic internally while the OpenShift route handles TLS termination. This is required to

prevent redirect loops (`ERR_TOO_MANY_REDIRECTS`) when using edge TLS termination.



The ArgoCD CR includes `extraConfig` with OIDC configuration for ETX Keycloak integration. This provides centralized authentication using the same credentials as GitLab, Quay, and Vault. The `argocd-oidc-secret` created earlier is referenced automatically.

Wait for Argo CD to be ready (this may take 3-5 minutes):

```
oc get pods -n etx-app-dev -w
```

Press Ctrl+C when you see all `argocd-*` pods showing `1/1 Ready`.

Verify Route Creation

The ArgoCD operator should automatically create a route. Verify it exists:

```
oc get route argocd-server -n etx-app-dev
```

If the route doesn't exist, create it manually:

```
oc create route edge argocd-server \
  --service=argocd-server \
  --port=http \
  --insecure-policy=Redirect \
  -n etx-app-dev
```

Access the Argo CD Dashboard

Get the Argo CD dashboard URL:

```
echo "https://$(oc get route argocd-server -n etx-app-dev -o jsonpath='{.spec.host}')
```

You can log into Argo CD using either the admin account or Keycloak SSO.

Option 1: ETX Keycloak SSO (Recommended)

1. Open the Argo CD URL in your browser
2. You'll be automatically redirected to ETX Keycloak login page
3. Log in with your ETX credentials:
 - `platform-admin` / `openshift` (admin access)
 - `platform-developer` / `openshift` (admin access)
4. After successful authentication, you'll be redirected back to the Argo CD dashboard



The OIDC integration provides automatic redirect to Keycloak for a streamlined login experience. No need to click a separate login button.

Option 2: Admin Account

Get the default admin password:

```
oc get secret argocd-cluster -n etx-app-dev \
-o jsonpath='{.data.admin\.password}' | base64 -d && echo
```

Open the Argo CD URL and log in with: * Username: **admin** * Password: (from above command)



Using Keycloak SSO is recommended as it provides centralized authentication with the same credentials used for GitLab, Quay, and Vault.

Troubleshooting: ERR_TOO_MANY_REDIRECTS

If you get **ERR_TOO_MANY_REDIRECTS** when accessing the Argo CD dashboard:

Cause: The Argo CD server is not configured for insecure mode behind a TLS-terminating proxy.

Solution:

1. Update the ArgoCD CR to add **server.insecure: true**:

```
oc patch argocd argocd -n etx-app-dev --type=merge -p
'{"spec":{"server":{"insecure":true}}}'
```

2. Wait for the argocd-server pod to restart (this happens automatically):

```
oc get pods -n etx-app-dev -l app.kubernetes.io/name=argocd-server -w
```

Press Ctrl+C when you see the new pod showing **1/1 Running**.

3. Verify the configuration was applied:

```
oc get argocd argocd -n etx-app-dev -o jsonpath='{.spec.server.insecure}{"\n"}'
```

Should output: **true**

4. Clear your browser cache or use incognito/private mode to avoid cached redirects
5. Try accessing the dashboard again:

```
echo "https://$(oc get route argocd-server -n etx-app-dev -o
```

```
jsonpath='{.spec.host}')
```



If you still see the error after following these steps, try a different browser or clear all site data for the ArgoCD URL. The browser may have cached the redirect loop.

Prepare the GitOps Repository

Create a GitOps repository in GitLab with the following structure. This repository holds the Kubernetes manifests that Argo CD will sync:

1. Log into GitLab (get the URL first):

```
# Get GitLab URL
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')
```

2. Create a new project: **etx-app-gitops** in the software-factory group
3. Clone it locally:

```
git clone ${GITLAB_URL}/software-factory/etx-app-gitops.git
cd etx-app-gitops
```

We will create the following directory structure in the GitOps repository:

```
etx-app-gitops/
├── environments/
│   ├── dev/
│   │   ├── namespace.yaml
│   │   ├── deployment.yaml
│   │   ├── service.yaml
│   │   └── route.yaml
│   └── stage/
│       ├── namespace.yaml
│       ├── deployment.yaml
│       ├── service.yaml
│       └── route.yaml
└── README.md
```

Dev Environment Manifests

Create `environments/dev/namespace.yaml`:

```
mkdir -p environments/dev
cat > environments/dev/namespace.yaml <<'EOF'
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: etx-app-dev
EOF
```

Create `environments/dev/deployment.yaml`:

```
cat > environments/dev/deployment.yaml <<'EOF'
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
  labels:
    app: etx-app-base-app
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true" # ①
spec:
  replicas: 1
  selector:
    matchLabels:
      app: etx-app-base-app
  template:
    metadata:
      labels:
        app: etx-app-base-app
      annotations:
        instrumentation.opentelemetry.io/inject-java: "true" # ①
    spec:
      containers:
        - name: etx-app-base-app
          image: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app:latest ②
          ports:
            - containerPort: 8080
          resources:
            requests:
              cpu: 250m
              memory: 512Mi
            limits:
              cpu: "1"
              memory: 1Gi
EOF
```

① Enables OpenTelemetry auto-instrumentation. The Instrumentation CR created in the [Supporting Services](#) section will inject the Java agent automatically.

② The CI pipeline updates this image tag after a successful build.

Create `environments/dev/service.yaml`:

```
cat > environments/dev/service.yaml <<'EOF'
apiVersion: v1
kind: Service
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
spec:
  selector:
    app: etx-app-base-app
  ports:
    - port: 8080
      targetPort: 8080
EOF
```

Create `environments/dev/route.yaml`:

```
cat > environments/dev/route.yaml <<'EOF'
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: etx-app-base-app
  namespace: etx-app-dev
spec:
  to:
    kind: Service
    name: etx-app-base-app
  port:
    targetPort: 8080
  tls:
    termination: edge
EOF
```

Commit and push the dev environment:

```
git add environments/dev/
git commit -m "feat: add dev environment manifests"
git push origin main
```

Create the Argo CD Application

Grant Argo CD permission to manage resources in the `etx-app-dev` namespace:

```
oc adm policy add-role-to-user admin \
  system:serviceaccount:etx-app-dev:argocd-application-controller \
```

```
-n etx-app-dev
```

Create an Argo CD Application that watches the **dev** environment.

First, get the GitLab URL:

```
# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "Using GitLab URL: $GITLAB_URL"
```

Then create the Application:

```
oc apply -f - <<EOF
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: etx-app-dev
  namespace: etx-app-dev
spec:
  project: default
  source:
    repoURL: ${GITLAB_URL}/software-factory/etx-app-gitops.git
    targetRevision: main
    path: environments/dev
  destination:
    server: https://kubernetes.default.svc
    namespace: etx-app-dev
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=false
EOF
```



CreateNamespace=false because the namespace already exists.

Verify the Deployment

Check that Argo CD has synced the application:

```
# Argo CD application status
oc get application etx-app-dev -n etx-app-dev \
-o jsonpath='{.status.sync.status}'{"\n"}'
```


The output should be **Synced**.

Verify the application is running:

```
oc get pods -n etx-app-dev -l app=etx-app-base-app
oc get route etx-app-base-app -n etx-app-dev -o jsonpath='{.spec.host}{"\n"}'
```

Open the route URL in your browser to confirm the application is accessible.

Verify OpenTelemetry Auto-Instrumentation

Check that the Java agent was injected into the application pod:

```
oc get pod -n etx-app-dev -l app=etx-app-base-app \
-o jsonpath='{.items[0].spec.initContainers[*].name}{"\n"}'
```

You should see an **opentelemetry-auto-instrumentation** init container.

Verify traces are being collected by checking the OTEL collector logs:

```
oc logs -n etx-app-dev -l app.kubernetes.io/name=otel-collector-collector --tail=50
```

You should see trace data in the logs.

Integrate CI and GitOps

The CI pipeline needs to update the image tag in the GitOps repository after a successful build. Add a final task to the pipeline that commits the new image tag:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: update-gitops-repo
  namespace: etx-app-dev
spec:
  params:
    - name: gitops-repo-url
      type: string
    - name: image-tag
      type: string
    - name: environment
      type: string
      default: dev
  workspaces:
    - name: source
  steps:
```

```

- name: update-image
  image: registry.access.redhat.com/ubi9/ubi:latest
  script: |
    #!/bin/bash
    set -e

    # Install git
    dnf install -y git && dnf clean all

    git clone $(params.gitops-repo-url) /tmp/gitops
    cd /tmp/gitops/environments/$(params.environment)

    # Update the image tag in the deployment manifest
    sed -i "s|image:.*etx-app-base-app:.*|image: image-registry.openshift-image-
registry.svc:5000/etx-app-dev/etx-app-base-app:$(params.image-tag)|" deployment.yaml

    git config user.email "pipeline@openshift.local"
    git config user.name "Tekton Pipeline"
    git add .
    git commit -m "Update etx-app-base-app image to $(params.image-tag)"
    git push
EOF

```

Update the pipeline to include this task at the end.

Add the **update-gitops** task to the pipeline spec. Get the GitLab URL first, then update the pipeline:

```

# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

# Add to your pipeline YAML under spec.tasks

```

Example task definition to add:

```

- name: update-gitops
  runAfter: [build-image]
  taskRef:
    name: update-gitops-repo
  params:
    - name: gitops-repo-url
      value: ${GITLAB_URL}/software-factory/etx-app-gitops.git
    - name: image-tag
      value: $(params.git-revision)
    - name: environment
      value: dev
  workspaces:
    - name: source

```

workspace: shared-workspace



Use shell variable expansion `${GITLAB_URL}` when creating the pipeline with a heredoc, or replace with the actual URL when editing the YAML file.

When the pipeline pushes the updated manifest, Argo CD detects the change and automatically syncs the new image to the cluster.

Multi-Cluster Promotion to Stage



The secondary cluster for staging has been pre-configured as part of the bootstrap process. Argo CD already has the necessary credentials to deploy to the staging cluster.

Verify Secondary Cluster Configuration

Check that Argo CD can see the secondary cluster.

Option 1: Using `oc` commands (Recommended)

```
# List cluster secrets in Argo CD
oc get secret -n etx-app-dev -l argocd.argoproj.io/secret-type=cluster

# View cluster details
oc get secret -n etx-app-dev -l argocd.argoproj.io/secret-type=cluster -o json | \
jq -r '.items[] | .metadata.name + " - " + (.data.name | @base64d)'
```

You should see secrets for: - In-cluster (local/primary cluster) - Secondary staging cluster (pre-configured by bootstrap)

Option 2: Using `argocd` CLI

If you have the `argocd` CLI installed:

```
# Login to Argo CD
argocd login $(oc get route argocd-server -n etx-app-dev -o jsonpath='{.spec.host}') \
--insecure --username admin \
--password $(oc get secret argocd-cluster -n etx-app-dev -o \
jsonpath='{.data.admin\.password}' | base64 -d)

# List clusters
argocd cluster list
```

You should see two clusters: - <https://kubernetes.default.svc> (local/primary cluster) - Secondary staging cluster (pre-configured by bootstrap)



The bootstrap automation created the necessary ServiceAccount on the staging

cluster and added it to Argo CD. For details on this configuration, see https://github.com/rh-etx-app-platform/etx_app_platform_bootstrap/issues/8

Create Stage Environment Manifests

Create stage environment manifests in your GitOps repository:

```
cd ~/workshop/etx-app-gitops
mkdir -p environments/stage
```

Copy the dev manifests as a template:

```
cp environments/dev/*.yaml environments/stage/
```

Update `environments/stage/namespace.yaml`:

```
cat > environments/stage/namespace.yaml <<'EOF'
apiVersion: v1
kind: Namespace
metadata:
  name: etx-app-staging
EOF
```

Update `environments/stage/deployment.yaml` to use the `etx-app-staging` namespace and Quay registry.

First, get the Quay URL:

```
# Get Quay URL dynamically
QUAY_URL=$(oc get route -n quay -l app=quay -o jsonpath='{.items[0].spec.host}')
echo "Using Quay URL: $QUAY_URL"
```

Then create the deployment manifest:

```
cat > environments/stage/deployment.yaml <<EOF
apiVersion: apps/v1
kind: Deployment
metadata:
  name: etx-app-base-app
  namespace: etx-app-staging
  labels:
    app: etx-app-base-app
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
spec:
```

```

replicas: 2
selector:
  matchLabels:
    app: etx-app-base-app
template:
  metadata:
    labels:
      app: etx-app-base-app
    annotations:
      instrumentation.opentelemetry.io/inject-java: "true"
  spec:
    containers:
      - name: etx-app-base-app
        image: ${QUAY_URL}/etx-app/etx-app-base-app:stable
        ports:
          - containerPort: 8080
        resources:
          requests:
            cpu: 500m
            memory: 1Gi
          limits:
            cpu: "2"
            memory: 2Gi

```

EOF

Key differences from dev:

- **Namespace:** `etx-app-staging` instead of `etx-app-dev`
- **Replicas:** Increased to 2 for higher availability
- **Image:** Uses Quay registry for cross-cluster image distribution instead of internal registry

Update `environments/stage/service.yaml` to use `etx-app-staging` namespace:

```

sed -i 's/namespace: etx-app-dev/namespace: etx-app-staging/'
environments/stage/service.yaml

```

Update `environments/stage/route.yaml` to use `etx-app-staging` namespace:

```

sed -i 's/namespace: etx-app-dev/namespace: etx-app-staging/'
environments/stage/route.yaml

```

Update `environments/stage/namespace.yaml`:

```

sed -i 's/name: etx-app-dev/name: etx-app-staging/' environments/stage/namespace.yaml

```

Verify the changes:

```
grep namespace environments/stage/*.yaml
```

Commit and push:

```
git add environments/stage/  
git commit -m "feat: add stage environment manifests"  
git push origin main
```

Create Stage Application

Create an Argo CD Application for the staging environment on the secondary cluster.

First, get the GitLab URL and the secondary cluster server URL:

```
# Get GitLab URL  
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o  
jsonpath='{.items[0].spec.host}')"  
echo "Using GitLab URL: $GITLAB_URL"  
  
# Get the staging cluster server URL from Argo CD Secret  
STAGING_CLUSTER=$(oc get secret -n etx-app-dev -l argocd.argoproj.io/secret-  
type=cluster -o json | \  
jq -r '.items[] | select(.metadata.annotations."cluster-name" | contains("cluster2")  
or contains("staging") or contains("secondary")) | .data.server' | \  
base64 -d)  
  
# If not found in secrets, try using argocd CLI  
if [ -z "$STAGING_CLUSTER" ]; then  
  # Login to Argo CD  
  argocd login $(oc get route argocd-server -n etx-app-dev -o jsonpath='{.spec.host}') \  
  --insecure --username admin \  
  --password $(oc get secret argocd-cluster -n etx-app-dev -o  
jsonpath='{.data.admin\.password}' | base64 -d)  
  
  # Get cluster URL  
  STAGING_CLUSTER=$(argocd cluster list -o json | jq -r '[] | select(.name != "in-  
cluster") | .server')  
fi  
  
echo "Staging cluster: ${STAGING_CLUSTER}"
```

Then create the Application:

```
oc apply -f - <<EOF  
apiVersion: argoproj.io/v1alpha1  
kind: Application
```

```

metadata:
  name: etx-app-stage
  namespace: etx-app-dev
spec:
  project: default
  source:
    repoURL: ${GITLAB_URL}/software-factory/etx-app-gitops.git
    targetRevision: main
    path: environments/stage
  destination:
    server: ${STAGING_CLUSTER}
    namespace: etx-app-staging
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
    syncOptions:
      - CreateNamespace=true
EOF

```



The staging cluster destination points to the secondary cluster that was pre-configured by the bootstrap automation.

Verify Stage Deployment

Check the stage application status:

```

oc get application etx-app-stage -n etx-app-dev \
-o jsonpath='{.status.sync.status}'

```

The output should be **Synced**.

View the application in Argo CD UI:

1. Open the Argo CD dashboard
2. You should see two applications:
 - **etx-app-dev** - Running on primary cluster
 - **etx-app-stage** - Running on secondary (staging) cluster
3. Click on **etx-app-stage** to view its deployment status



Since Argo CD is managing the staging cluster remotely, you can view the application status through the Argo CD UI without needing to login to the secondary cluster directly.

Promotion Workflow Overview

The promotion workflow from dev to staging follows these steps:

1. CI pipeline builds and pushes image to internal registry with commit SHA tag
2. Image is promoted to Quay registry with a **stable** tag (covered in [Promotion Pipeline](#))
3. GitOps repository `environments/stage/deployment.yaml` is updated with new image tag
4. Argo CD detects the change and automatically syncs to the secondary cluster

This GitOps approach ensures:

- **Version control** - All deployment changes are tracked in Git
- **Auditability** - Full history of what was deployed when
- **Rollback capability** - Revert to previous commit to rollback
- **Multi-cluster consistency** - Same manifests can deploy to multiple clusters

Quick Verification

Verify the complete deployment:

```
# Check Argo CD application status
oc get application etx-app-dev -n etx-app-dev -o jsonpath='{.status.sync.status}:
{.status.health.status}{"\n"}'

# Check application pods
oc get pods -n etx-app-dev -l app=etx-app-base-app

# Check application health endpoint
curl http://$(oc get route etx-app-base-app -n etx-app-dev -o
jsonpath='{.spec.host}')/q/health

# Check OpenTelemetry injection
oc get pod -n etx-app-dev -l app=etx-app-base-app \
-o jsonpath='{.items[0].spec.initContainers[*].name}{"\n"}'
```

Expected output should show: **Synced: Healthy** for Argo CD, **Running** pods, and **opentelemetry-auto-instrumentation** init container.



Common Issues:

- **ImagePullBackOff:** Image not in registry - run CI pipeline first
- **CrashLoopBackOff:** Check pod logs - often missing Kafka or DB connection
- **Argo CD Out of Sync:** Click "Sync" in Argo CD UI or check GitOps repo for errors
- **Health check failing:** Verify all dependencies (DB, Kafka) are running

Summary

You have completed the Application Deployment exercise. Your application now has:

- Your own Argo CD instance for GitOps deployments
- GitOps repository structure (dev and stage environments)
- Automated sync with self-healing enabled
- OpenTelemetry auto-instrumentation for distributed tracing
- Integration between CI pipeline and GitOps repository
- Multi-cluster deployment capability (dev → stage)

What's Next

The current setup deploys to dev automatically when the CI pipeline completes. To promote images from dev to staging, you need a promotion pipeline.

Proceed to [Promotion Pipeline](#) to learn how to:

- Copy images from internal registry to Quay
- Update GitOps repository for staging environment
- Trigger automatic deployment to the secondary cluster via Argo CD

Lab 3

Overview

In the previous exercises, you set up a CI pipeline that builds container images and deploys them to the dev environment automatically. Now you'll create a promotion pipeline that moves tested images from dev to staging on the secondary cluster.

This promotion pipeline:

- Copies container images from the internal registry to Quay (for cross-cluster access)
- Updates the GitOps repository with the new image tag for staging
- Triggers Argo CD to sync the staging environment on the secondary cluster

Prerequisites

Before starting this exercise, ensure you have completed:

- [Application CI Pipeline](#)
- [Application Deployment](#)

You should have:

- A working CI pipeline that builds and pushes images

- Argo CD managing both dev and stage environments
- Access to Quay registry (pre-configured by bootstrap)

Understanding the Promotion Flow

The promotion workflow follows these steps:

Table 1. Promotion Sequence

Step	Component	Action
1	Developer	Approves promotion (manual trigger)
2	GitLab	Triggers promotion pipeline
3	Promotion Pipeline	Pulls image from internal registry by SHA
4	Promotion Pipeline	Pushes image to Quay with 'stable' tag
5	Promotion Pipeline	Updates <code>environments/stage/deployment.yaml</code> in GitOps repo
6	Argo CD	Detects change in GitOps repository
7	Argo CD	Syncs new image to staging cluster

Key points:

- **Dev environment** uses images from internal registry (fast, local)
- **Staging environment** uses images from Quay (cross-cluster accessible)
- **Promotion is explicit** - you decide when code is ready for staging
- **GitOps ensures** staging deployments are version-controlled

Create Promotion Pipeline Resources

Pipeline Workspace

The promotion pipeline needs a workspace for cloning the GitOps repository:

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: promotion-workspace-pvc
  namespace: etx-app-dev
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 2Gi
EOF
```

Fetch Quay Credentials from Vault

Create a task to retrieve Quay credentials from Vault:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: vault-fetch-quay-creds
  namespace: etx-app-dev
spec:
  results:
    - name: quay-server
    - name: quay-username
    - name: quay-password
  steps:
    - name: fetch
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        set -e

        # Install curl and jq
        microdnf install -y curl jq && microdnf clean all

        # Authenticate to Vault using ServiceAccount token
        VAULT_ADDR="{vault_url}"
        SA_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

        VAULT_TOKEN=$(curl -s --request POST \
          "${VAULT_ADDR}/v1/auth/kubernetes/login" \
          --data "{\"role\":\"pipeline\",\"jwt\":\"${SA_TOKEN}\"}" \
          | jq -r '.auth.client_token')

        # Fetch Quay credentials
        CREDS=$(curl -s --header "X-Vault-Token: ${VAULT_TOKEN}" \
          "${VAULT_ADDR}/v1/secret/data/registry/quay")

        echo "${CREDS}" | jq -r '.data.data.server' \
          | tr -d '\n' > $(results.quay-server.path)
        echo "${CREDS}" | jq -r '.data.data.username' \
          | tr -d '\n' > $(results.quay-username.path)
        echo "${CREDS}" | jq -r '.data.data.password' \
          | tr -d '\n' > $(results.quay-password.path)
EOF
```

Copy Image to Quay

Create a task that copies the image from the internal registry to Quay using Skopeo:

```

oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: copy-image-to-quay
  namespace: etx-app-dev
spec:
  params:
    - name: source-image
      type: string
      description: Source image URL (internal registry)
    - name: dest-image
      type: string
      description: Destination image URL (Quay)
    - name: dest-username
      type: string
    - name: dest-password
      type: string
  steps:
    - name: copy
      image: quay.io/skopeo/stable:latest
      script: |
        #!/bin/bash
        set -e

        echo "Copying image from internal registry to Quay..."
        echo "Source: $(params.source-image)"
        echo "Destination: $(params.dest-image)"

        # Copy image using skopeo
        skopeo copy \
          --src-tls-verify=false \
          --dest-tls-verify=false \
          --dest-creds="$(params.dest-username):$(params.dest-password)" \
          "docker://$(params.source-image)" \
          "docker://$(params.dest-image)"

        echo "Image copied successfully!"
EOF

```

Update GitOps Repository

Create a task to update the staging deployment manifest with the new image tag:

```

oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: update-gitops-staging

```

```

namespace: etx-app-dev
spec:
  params:
    - name: gitops-repo-url
      type: string
    - name: image-url
      type: string
      description: Full image URL to update in staging manifest
    - name: git-username
      type: string
      default: "Tekton Pipeline"
    - name: git-email
      type: string
      default: "pipeline@openshift.local"
  workspaces:
    - name: source
  steps:
    - name: update
      image: registry.access.redhat.com/ubi9/ubi:latest
      script: |
        #!/bin/bash
        set -e

        # Install git
        dnf install -y git && dnf clean all

        # Clone GitOps repo
        git clone $(params.gitops-repo-url) /workspace/gitops
        cd /workspace/gitops

        # Update the image in staging deployment
        sed -i "s|image:.*etx-app-base-app:.*|image: $(params.image-url)|" \
          environments/stage/deployment.yaml

        # Commit and push
        git config user.email "$(params.git-email)"
        git config user.name "$(params.git-username)"
        git add environments/stage/deployment.yaml
        git commit -m "Promote to staging: $(params.image-url)"
        git push origin main

        echo "GitOps repository updated successfully!"
EOF

```

Create the Promotion Pipeline

Now combine all tasks into a promotion pipeline:

```

oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1

```

```

kind: Pipeline
metadata:
  name: etx-app-promotion
  namespace: etx-app-dev
spec:
  params:
    - name: source-image-tag
      type: string
      description: Image tag to promote (e.g., commit SHA)
    - name: dest-image-tag
      type: string
      default: stable
      description: Tag to use in Quay (e.g., stable, v1.0.0)
    - name: gitops-repo-url
      type: string
      default: {gitlab_url}/software-factory/etx-app-gitops.git
  workspaces:
    - name: shared-workspace
  tasks:
    - name: fetch-quay-creds
      taskRef:
        name: vault-fetch-quay-creds

    - name: copy-to-quay
      runAfter: [fetch-quay-creds]
      taskRef:
        name: copy-image-to-quay
      params:
        - name: source-image
          value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app:${params.source-image-tag}
        - name: dest-image
          value: ${tasks.fetch-quay-creds.results.quay-server}/etx-app/etx-app-base-
app:${params.dest-image-tag}
        - name: dest-username
          value: ${tasks.fetch-quay-creds.results.quay-username}
        - name: dest-password
          value: ${tasks.fetch-quay-creds.results.quay-password}

    - name: update-gitops
      runAfter: [copy-to-quay]
      taskRef:
        name: update-gitops-staging
      params:
        - name: gitops-repo-url
          value: ${params.gitops-repo-url}
        - name: image-url
          value: ${tasks.fetch-quay-creds.results.quay-server}/etx-app/etx-app-base-
app:${params.dest-image-tag}
      workspaces:
        - name: source

```

```
workspace: shared-workspace
EOF
```

Apply the pipeline:

```
oc apply -f etx-app-promotion-pipeline.yaml
```

Test the Promotion Pipeline

First, get the GitLab URL dynamically:

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "GitLab URL: $GITLAB_URL"
```

Get a Valid Image Tag

Find a recent image tag from your dev environment:

```
# List recent images from dev CI pipeline
oc get imagestream etx-app-base-app -n etx-app-dev -o json | \
jq -r '.status.tags[].tag' | head -5
```

Pick one of the commit SHA tags from the output.

Run the Promotion Pipeline

Trigger the promotion pipeline manually.

First, get an actual commit SHA from the images built by the CI pipeline:

```
oc get images -n etx-app-dev | grep etx-app-base-app
```

Then create a PipelineRun (replace `<COMMIT_SHA>` with an actual image tag from above):

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-promotion-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-promotion
  params:
```

```
- name: source-image-tag
  value: <COMMIT_SHA>
- name: dest-image-tag
  value: stable
- name: gitops-repo-url
  value: ${GITLAB_URL}/software-factory/etx-app-gitops.git
workspaces:
- name: shared-workspace
  persistentVolumeClaim:
    claimName: promotion-workspace-pvc
taskRunTemplate:
  serviceAccountName: pipeline
EOF
```

Watch the pipeline logs. You should see:

1. Vault credentials retrieved
2. Image copied from internal registry to Quay
3. GitOps repository updated

Verify the Promotion

Check that the GitOps repository was updated:

```
cd ~/workshop/etx-app-gitops
git pull origin main
cat environments/stage/deployment.yaml | grep image:
```

You should see the Quay image URL with the **stable** tag.

Check the Argo CD application status:

```
oc get application etx-app-stage -n etx-app-dev -o
jsonpath='{.status.sync.status}'
```

The output should be **Synced**.

View in Argo CD UI:

1. Open the Argo CD dashboard
2. Click on the **etx-app-stage** application
3. You should see it's synced and healthy
4. The deployment should show the new image from Quay

Automate Promotion with Approval

For a more realistic workflow, you can add a manual approval step before promotion.

Option 1: Manual Trigger (Current Approach)

The current setup requires manually triggering the promotion pipeline. This is suitable for:

- Small teams
- Infrequent deployments
- High-touch release processes

Option 2: Automatic with Manual Approval

Add a manual approval task using Tekton:

```
oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: manual-approval
  namespace: etx-app-dev
spec:
  description: |
    Pauses pipeline execution until manual approval.
    Requires operator intervention to continue.
  steps:
    - name: wait-for-approval
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        echo "⏸ Pipeline paused - waiting for manual approval"
        echo "Run: oc patch taskrun <taskrun-name> -p
'{"spec\":{\"status\":"Succeeded\"}}'"
        sleep infinity
EOF
```

Add this task as the first step in your promotion pipeline to require approval before promoting to staging.

Option 3: GitLab Merge Request Trigger

Create a webhook that triggers the promotion pipeline when a merge request is merged:

1. Create a promotion branch in your application repo
2. Open a merge request for promotion
3. Add reviewers for approval
4. On merge, trigger the promotion pipeline via GitLab webhook

This approach provides:

- Code review for promotion decisions
- Audit trail in Git
- Team collaboration

Rollback Strategy

If you need to rollback a staging deployment:

Method 1: Revert Git Commit

```
cd ~/workshop/etx-app-gitops
git log environments/stage/deployment.yaml
git revert <commit-hash>
git push origin main
```

Argo CD will automatically sync the previous version.

Method 2: Update to Previous Tag

Run the promotion pipeline with a previous image tag:

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-promotion-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-promotion
  params:
    - name: source-image-tag
      value: <PREVIOUS_SHA>
    - name: dest-image-tag
      value: stable
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: promotion-workspace-pvc
  taskRunTemplate:
    serviceAccountName: pipeline
EOF
```

Summary

You have completed the Promotion Pipeline exercise. You now have:

- Automated image promotion from internal registry to Quay
- GitOps-based staging deployments
- Secure credential management via Vault
- Multi-cluster deployment capability
- Rollback capability through Git history

Key Takeaways

Promotion != Re-build:

- The same container image (by SHA) is promoted from dev to staging
- No code changes between environments - only configuration
- This ensures you test exactly what you deploy

GitOps ensures consistency:

- All deployment state is in Git
- Changes are auditable and reversible
- Argo CD provides automatic sync and drift detection

Multi-cluster architecture:

- Dev uses internal registry (fast, local)
- Staging uses Quay (cross-cluster accessible)
- Argo CD manages both from single control plane

Next Steps

Proceed to [Lab 2: Application Modernization \(MTA\)](#) to learn how to modernize legacy Java applications.

Lab 4

Overview

Create a Tekton CI pipeline that clones your application source, builds and tests it, retrieves registry credentials from Vault, and pushes the container image. Then wire up Tekton Triggers to fire the pipeline automatically when code is pushed to GitLab.

Pipeline Workspace and ServiceAccount

Create a workspace PVC and a ServiceAccount for the pipeline:

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pipeline-workspace-pvc
  namespace: etx-app-dev
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: pipeline
  namespace: etx-app-dev
EOF
```

Expected output:

```
persistentvolumeclaim/pipeline-workspace-pvc created
serviceaccount/pipeline created
```



If the ServiceAccount already exists (created during namespace setup), you may see `serviceaccount/pipeline configured` instead of `created`, along with a warning about a missing annotation. This is normal - `oc apply` automatically patches the annotation, and the resource is correctly configured.

Grant ServiceAccount Permissions

The pipeline ServiceAccount needs permission to push images and manage resources in the namespace:

```
oc adm policy add-role-to-user edit \
  system:serviceaccount:etx-app-dev:pipeline \
  -n etx-app-dev
```

This grants the `edit` role, allowing the pipeline to create ImageStreams, Deployments, and other resources.

Retrieve Registry Credentials from Vault

Create a Tekton Task that authenticates to Vault using the Kubernetes auth method configured in the [Supporting Services](#) section and writes registry credentials to the workspace.

First, get the Vault URL from the cluster:

```
# Get Vault URL dynamically
VAULT_URL="https://$(oc get route -n vault -o jsonpath='{.items[0].spec.host}')"
echo "Using Vault URL: $VAULT_URL"
```

Then create the Task:

```
oc apply -f - <<EOF
apiVersion: tekton.dev/v1
kind: Task
metadata:
  name: vault-fetch-registry-creds
  namespace: etx-app-dev
spec:
  workspaces:
    - name: source
  results:
    - name: registry-server
    - name: registry-username
    - name: registry-password
  steps:
    - name: fetch
      image: registry.access.redhat.com/ubi9/ubi-minimal:latest
      script: |
        #!/bin/sh
        set -e

        # Install jq
        microdnf install -y jq && microdnf clean all

        # Authenticate to Vault using ServiceAccount token
        VAULT_ADDR="${VAULT_URL}"
        SA_TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)

        VAULT_TOKEN=$(curl -s --request POST \
          "\${VAULT_ADDR}/v1/auth/kubernetes/login" \
          --data '{"role\":\"pipeline\",\"jwt\":\"${SA_TOKEN}\"}' \
          | jq -r '.auth.client_token')

        # Fetch registry credentials
        CRED=$(curl -s --header "X-Vault-Token: \${VAULT_TOKEN}" \
          "\${VAULT_ADDR}/v1/secret/data/registry")
```

```

echo "\${CREDS}" | jq -r '.data.data.server' \
| tr -d '\n' > \$(results.registry-server.path)
echo "\${CREDS}" | jq -r '.data.data.username' \
| tr -d '\n' > \$(results.registry-username.path)
echo "\${CREDS}" | jq -r '.data.data.password' \
| tr -d '\n' > \$(results.registry-password.path)

```

EOF

Create the Pipeline

Define the CI pipeline with four tasks: clone, build/test, fetch credentials, and build/push the container image:

```

oc apply -f - <<'EOF'
apiVersion: tekton.dev/v1
kind: Pipeline
metadata:
  name: etx-app-base-app-ci
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
      type: string
    - name: git-revision
      type: string
      default: main
    - name: image-name
      type: string
  workspaces:
    - name: shared-workspace
  tasks:
    - name: clone
      taskRef:
        resolver: cluster
        params:
          - name: kind
            value: task
          - name: name
            value: git-clone
          - name: namespace
            value: openshift-pipelines
      params:
        - name: URL
          value: \$(params.git-url)
        - name: REVISION
          value: \$(params.git-revision)
      workspaces:
        - name: output
          workspace: shared-workspace

```

```

- name: build-and-test
  runAfter: [clone]
  taskRef:
    resolver: cluster
    params:
      - name: kind
        value: task
      - name: name
        value: maven
      - name: namespace
        value: openshift-pipelines
  params:
    - name: GOALS
      value: ["package", "-DskipTests=false"]
  workspaces:
    - name: source
      workspace: shared-workspace

- name: fetch-creds
  runAfter: [clone]
  taskRef:
    name: vault-fetch-registry-creds
  workspaces:
    - name: source
      workspace: shared-workspace

- name: build-image
  runAfter: [build-and-test, fetch-creds]
  taskRef:
    resolver: cluster
    params:
      - name: kind
        value: task
      - name: name
        value: buildah
      - name: namespace
        value: openshift-pipelines
  params:
    - name: IMAGE
      value: $(params.image-name):$(params.git-revision)
    - name: DOCKERFILE
      value: Containerfile
  workspaces:
    - name: source
      workspace: shared-workspace

```

EOF



`git-clone`, `maven`, and `buildah` are standard Tasks installed in the `openshift-pipelines` namespace. This pipeline resolves them with the cluster resolver

(replacing deprecated `ClusterTask` references). Run `oc get task -n openshift-pipelines` to verify they are available.

Test the Pipeline Manually

Run the pipeline once to verify it works before adding webhooks.

First, get the GitLab URL from the cluster:

```
# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "Using GitLab URL: $GITLAB_URL"
```

Then create the PipelineRun:

```
oc create -f - <<EOF
apiVersion: tekton.dev/v1
kind: PipelineRun
metadata:
  generateName: etx-app-base-app-ci-
  namespace: etx-app-dev
spec:
  pipelineRef:
    name: etx-app-base-app-ci
  params:
    - name: git-url
      value: ${GITLAB_URL}/software-factory/etx_app_base_app
    - name: git-revision
      value: main
    - name: image-name
      value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-app-
base-app
  workspaces:
    - name: shared-workspace
      persistentVolumeClaim:
        claimName: pipeline-workspace-pvc
  taskRunTemplate:
    serviceAccountName: pipeline
EOF
```

Expected output:

```
pipelinerun.tekton.dev/etx-app-base-app-ci-xxxxx created
```

Monitor the PipelineRun:


```
# List all PipelineRuns
oc get pipelinerun -n etx-app-dev
```

You should see your PipelineRun with status **Running** or **Succeeded**.

Monitor Pipeline Execution

There are several ways to monitor the pipeline:

Option 1: Watch PipelineRun status (Recommended)

```
# Watch PipelineRun status in real-time (press Ctrl+C to exit)
oc get pipelinerun -n etx-app-dev -w
```

This shows the overall status of all PipelineRuns and updates in real-time.

Option 2: View individual Task logs

```
# Get the name of the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

echo "PipelineRun: $LATEST_PR"

# List all TaskRuns for this PipelineRun
oc get taskrun -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR
```

To view logs of a specific TaskRun, use the name from the list above:

```
# View logs of a specific TaskRun (example: fetch-creds)
oc logs -n etx-app-dev -l tekton.dev/taskRun=etx-app-base-app-ci-xxxxx-fetch-creds -f
```

Or automatically get logs from a failed TaskRun:

```
# Get the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

# Get the first failed TaskRun
FAILED_TASK=$(oc get taskrun -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR -o
jsonpath='{.items[?(@.status.conditions[0].reason=="Failed")].metadata.name}' | awk
'{print $1}')

if [ -n "$FAILED_TASK" ]; then
  echo "Viewing logs for failed TaskRun: $FAILED_TASK"
  oc logs -n etx-app-dev -l tekton.dev/taskRun=$FAILED_TASK
```

```
else
  echo "No failed TaskRuns found"
fi
```

Option 3: View logs of currently running pod

```
# Get the name of the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

echo "Watching logs for PipelineRun: $LATEST_PR"

# Follow the logs of the currently running pod
oc logs -f $(oc get pods -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR --sort
-by=.metadata.creationTimestamp -o jsonpath='{.items[-1].metadata.name}') -n etx-app-
dev
```



Option 3 only shows logs from one Task at a time. When that Task completes, the command exits. Use Option 1 for continuous monitoring or Option 2 to view specific Task logs.

Watch the logs to confirm all tasks complete successfully.

Troubleshooting Common Issues

fetch-creds task fails with "parse error: Invalid numeric literal"

This error means the Vault response is not valid JSON. Common causes:

- **Vault is unavailable:** Check if Vault pods are running:

```
oc get pods -n vault
```

If Vault is down, wait for it to recover and retry the pipeline.

- **Vault authentication not configured:** Verify the Kubernetes auth method exists in Vault UI at the vault URL.
- **ServiceAccount lacks permissions:** Verify the pipeline ServiceAccount exists:

```
oc get serviceaccount pipeline -n etx-app-dev
```

Pipeline succeeds but image not found in registry

Verify the image was pushed:

```
oc get imagestream etx-app-base-app -n etx-app-dev
```

```
oc get imagestreamtag etx-app-base-app:main -n etx-app-dev
```

git-clone task fails with "Could not resolve host"

The GitLab URL is incorrect. Verify:

```
oc get route -n gitlab -l app=webservice -o jsonpath='{.items[0].spec.host}'{"\n"}
```

Recreate the PipelineRun with the correct GitLab URL.

Set Up Webhooks with Tekton Triggers

Create the trigger resources so the pipeline runs automatically when code is pushed to GitLab.

TriggerBinding

Maps fields from the GitLab webhook payload to pipeline parameters:

```
oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerBinding
metadata:
  name: gitlab-push-binding
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
      value: $(body.repository.git_http_url)
    - name: git-revision
      value: $(body.checkout_sha)
EOF
```

TriggerTemplate

Creates a PipelineRun when triggered:

```
oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerTemplate
metadata:
  name: gitlab-push-template
  namespace: etx-app-dev
spec:
  params:
    - name: git-url
    - name: git-revision
  resourcetemplates:
```

```

- apiVersion: tekton.dev/v1
  kind: PipelineRun
  metadata:
    generateName: etx-app-base-app-ci-
  spec:
    pipelineRef:
      name: etx-app-base-app-ci
    params:
      - name: git-url
        value: $(tt.params.git-url)
      - name: git-revision
        value: $(tt.params.git-revision)
      - name: image-name
        value: image-registry.openshift-image-registry.svc:5000/etx-app-dev/etx-
app-base-app
    workspaces:
      - name: shared-workspace
        persistentVolumeClaim:
          claimName: pipeline-workspace-pvc
    taskRunTemplate:
      serviceAccountName: pipeline
EOF

```

EventListener

Exposes an HTTP endpoint that GitLab can send webhooks to:

```

oc apply -f - <<'EOF'
apiVersion: triggers.tekton.dev/v1beta1
kind: EventListener
metadata:
  name: gitlab-listener
  namespace: etx-app-dev
spec:
  serviceAccountName: pipeline
  triggers:
    - name: gitlab-push
      bindings:
        - ref: gitlab-push-binding
      template:
        ref: gitlab-push-template
EOF

```

Expose the EventListener

Create a route with TLS so GitLab can reach the EventListener securely:

```

oc create route edge el-gitlab-listener \
  --service=el-gitlab-listener \

```

```
--port=http-listener \
-n etx-app-dev
```



GitLab requires HTTPS for webhooks, so we create a route with edge TLS termination. If the route already exists, delete it first with `oc delete route el-gitlab-listener -n etx-app-dev` and recreate with TLS.

Get the webhook URL:

```
echo "https://$(oc get route el-gitlab-listener -n etx-app-dev -o
jsonpath='{.spec.host}')
```

Copy this URL for the next step.

Expected format:

```
https://el-gitlab-listener-etx-app-dev.apps.cluster-xxxxx.dynamic.redhatworkshops.io
```

Configure GitLab Webhook



Configuring webhooks in GitLab requires **Maintainer** or **Owner** role on the project. The `platform-developer` user does not have sufficient permissions. Use the `platform-admin` account or ask your instructor to grant you Maintainer access.

Step 1: Enable Webhooks to Local Network (REQUIRED)



GitLab blocks webhooks to local/internal networks by default for security (SSRF protection). Since the EventListener runs in the same cluster as GitLab, you **must** enable this feature or webhook creation will fail with "Invalid url given" error.

1. Get the GitLab Admin URLs and root password:

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
GITLAB_ROOT_PASSWORD=$(oc get secret -n gitlab gitlab-gitlab-initial-root-password
-o jsonpath='{.data.password}' | base64 -d)

echo "GitLab URL: $GITLAB_URL"
echo "Admin Area: $GITLAB_URL/admin"
echo "Network Settings (Direct): $GITLAB_URL/admin/application_settings/network"
echo ""
echo "Root Username: root"
echo "Root Password: $GITLAB_ROOT_PASSWORD"
```

2. Log into GitLab as root administrator:

- Open the GitLab URL in your browser
- Click **Standard** tab (not ETX Lab SSO)
- **Username:** `root`
- **Password:** Use the password from the command above



The `root` user is the GitLab built-in administrator. The `platform-admin` user from Keycloak does not have admin privileges by default.

1. Access the Admin Area:

Option A - Direct URL (Recommended):

Open the admin URL directly (from the command above):

```
{gitlab_url}/admin/application_settings/network
```

This takes you directly to the Network settings page.

Option B - UI Navigation:

- After logging in, look for the **wrench icon**  in the top navigation bar (usually far right)
- OR click your **avatar/profile picture** (top right) → select **Admin Area**
- If you don't see these options, verify you're logged in as `platform-admin`

Then navigate:

- In the left sidebar, find **Settings** section
- Click **Settings** → **Network**

2. Expand the **Outbound requests** section

3. Enable local network webhooks:

- Check ☒ **Allow requests to the local network from webhooks and integrations**
- Optionally check ☒ **Allow requests to the local network from system hooks**

4. Click btn:[Save changes] at the bottom

Finding the Admin Area:

The Admin Area is only accessible to users with administrator privileges. When logged in as `root`, you should see:



- A wrench icon () in the top navigation bar
- "Admin Area" option when clicking your avatar

If you don't see these, verify:

- You're logged in as **root** (GitLab built-in admin)
- You used the Standard login (not ETX Lab SSO)
- You're using the correct password from the OpenShift secret

Why not platform-admin?

The **platform-admin** user from Keycloak is not a GitLab administrator by default. To grant admin privileges to **platform-admin**:

1. Log in as **root**
2. Go to menu:Admin Area[Users]
3. Find the **platform-admin** user
4. Click **Edit** → Check **Admin** → **Save changes**

Direct URL: You can always access admin settings directly at: **{gitlab_url}/admin/application_settings/network**

Verification: After saving, you should see a green checkmark or success message indicating "Allow requests to the local network from webhooks and integrations" is enabled.



This setting is cluster-wide and affects all GitLab projects. In production environments, consider using external webhooks, GitLab Runner with allowedHosts, or more restrictive network policies.

Step 2: Add the Webhook to Your Repository



Configuring webhooks requires **Maintainer** or **Owner** role on the project (different from GitLab Admin). You can use **platform-admin** from Keycloak if it has Maintainer access to the project, or log in as **root**.

Option A: Use root account (if you're already logged in from Step 1)

1. Navigate to your project:
 - Click **Projects** → **Explore projects** → search for **etx_app_base_app**
 - Or navigate directly: **{gitlab_url}/software-factory/etx_app_base_app**

Option B: Use platform-admin account

1. Log out from **root** and log in via **ETX Lab SSO** as **platform-admin**
2. Navigate to your project:
 - **Projects** → **software-factory** → **etx_app_base_app**
3. If you see "Settings → Webhooks", you have Maintainer access and can proceed
4. If you don't see "Settings → Webhooks":
 - Ask your instructor to grant Maintainer role, OR

- Use the **root** account (Option A above)

Continue with either option:

1. Go to menu:Settings[Webhooks]
2. Click btn:[Add new webhook]
3. Configure the webhook:
 - **URL:** Paste the EventListener route URL:

```
echo "https://$(oc get route el-gitlab-listener -n etx-app-dev -o
jsonpath='{.spec.host}')
```

- **Trigger:** Check ☒ **Push events**
 - **SSL verification:** Uncheck for lab environments with self-signed certificates
4. Click btn:[Add webhook]
 5. Test the webhook:
 - Scroll down to **Project Hooks**
 - Find your webhook in the list
 - Click btn:[Test] → **Push events**
 - You should see "Hook executed successfully: HTTP 200" or similar



If you get "SSL certificate problem" error during testing, make sure "Enable SSL verification" is unchecked in the webhook configuration.

Using platform-developer Account:

If you want to use your **platform-developer** account instead of **platform-admin**, ask your instructor to grant you Maintainer role:



1. In GitLab (as admin), navigate to **software-factory/etx_app_base_app**
2. Go to menu:Settings[Members]
3. Find the user and change role to **Maintainer**
4. The user can now access Settings → Webhooks

Troubleshooting Webhook Issues

Error: "Invalid url given"

This error occurs when GitLab blocks webhooks to local/internal networks. Solutions:

1. **Enable local network webhooks** (required):
 - Go to menu:Admin Area[Settings > Network > Outbound requests]

- Check "Allow requests to the local network from webhooks and integrations"
- Save changes and try again

2. Verify the URL format:

- Must start with `https://` (not `http://`)
- Run: `echo "https://$(oc get route el-gitlab-listener -n etx-app-dev -o jsonpath='{.spec.host}')"`

3. Test EventListener is responding:

```
curl -I https://$(oc get route el-gitlab-listener -n etx-app-dev -o jsonpath='{.spec.host}')
```

Expected: HTTP 400 Bad Request (this is normal - it expects a webhook payload)

Error: "SSL certificate problem"

The cluster uses self-signed certificates. In GitLab webhook settings:

- Uncheck "Enable SSL verification"
- This is safe for lab environments

Test event fails with error

Check the EventListener pod is running:

```
oc get pods -n etx-app-dev | grep el-gitlab-listener
oc logs -n etx-app-dev -l eventlistener=gitlab-listener
```

Verify the Pipeline

Push a change to your repository and confirm the pipeline triggers:

```
# Make a trivial change
echo "# CI/CD test" >> README.md
git add README.md
git commit -m "Test webhook trigger"
git push origin feature/add-devfile
```

Check for new pipeline runs:

```
oc get pipelinerun -n etx-app-dev
```

You should see a new `etx-app-base-app-ci-*` PipelineRun triggered by the webhook.

Follow the latest run logs:

```
# Get the name of the latest PipelineRun
LATEST_PR=$(oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp -o
jsonpath='{.items[-1].metadata.name}')

echo "Following logs for: $LATEST_PR"

# Follow the logs
oc logs -f $(oc get pods -n etx-app-dev -l tekton.dev/pipelineRun=$LATEST_PR --sort
-by=.metadata.creationTimestamp -o jsonpath='{.items[-1].metadata.name}') -n etx-app-
dev
```

Verify the image was pushed:

```
oc get imagestream etx-app-base-app -n etx-app-dev
```

You should see the image tagged with the commit SHA.

Quick Verification

Verify all components are working:

```
# Check pipeline exists
oc get pipeline etx-app-base-app-ci -n etx-app-dev

# Check EventListener is running
oc get pods -n etx-app-dev | grep el-gitlab-listener

# Check webhook URL is exposed
oc get route el-gitlab-listener -n etx-app-dev

# Check recent pipeline runs
oc get pipelinerun -n etx-app-dev --sort-by=.metadata.creationTimestamp | tail -5
```



Common Issues:

- **Pipeline fails at clone:** Check Git credentials/authentication
- **Build fails:** Verify Maven build works locally first
- **Push fails:** Confirm ServiceAccount has **edit** role
- **Webhook not triggering:** Verify EventListener pod is Running and route is accessible

Summary

You now have:

- Tekton CI pipeline with Vault secrets integration
- Automated builds triggered by GitLab webhooks
- Container images pushed to OpenShift internal registry
- Maven-based Java/Quarkus build and test automation

Next Steps

Proceed to [Application Deployment with Argo CD](#).